Exit Ticket: Product Inventory CRUD

Question: explain when get(), filter(), save(), update(), and delete() execute database work.

My answer

get() runs the query right away and gives back one object. If nothing matches, it raises DoesNotExist.

filter() does not touch the database when I write it. It only builds a QuerySet. The query runs later, when I actually use the result, for example in a for loop, list(), len(), or print().

save() runs right away. It does an INSERT if the object is new, or an UPDATE if it already exists. With update_fields, only the fields I list are written.

update() runs right away as one UPDATE on all the rows that match the filter, and it returns how many rows changed.

delete() runs right away as a DELETE on the matching rows, and it returns how many were deleted.

Summary

Method	When it runs on the database	Example from the lab
get()	Immediately. Returns one object or raises DoesNotExist.	Product.objects.get(pk=1)
filter()	Lazy. Only when the QuerySet is evaluated (loop, list, len, print).	Product.objects.filter(is_active=True, stock__gt=0)
save()	Immediately. INSERT if new, UPDATE if it exists.	product.save(update_fields=["price"])
update()	Immediately. One UPDATE on all matching rows.	filter(category="Electronics").update(is_active=False)
delete()	Immediately. DELETE on the matching rows.	filter(is_active=False, stock=0).delete()

Main idea

all() and filter() only build the query. get(), save(), update() and delete() execute database work immediately.

How I checked it in the shell

I counted the queries to see when filter() really hits the database:

```
from django.db import connection
from django.test.utils import CaptureQueriesContext

with CaptureQueriesContext(connection) as ctx:
    qs = Product.objects.filter(is_active=True)
```

```
    print("after filter:", len(ctx))      # 0
    list(qs)
    print("after evaluating:", len(ctx))  # 1
```

The count was 0 after filter() and 1 after I evaluated the QuerySet, so filter() is lazy.